

7 These types of questions will help you isolate the problem layer, and are necessary
7 to determine the problem source. Remember that just because one layer is
7 reporting a problem, it does not always mean the root cause exists there.

7 Part of identifying where a problem is occurring is understanding the environment
7 in which it exists. You should always take some time to completely describe the
7 problem environment, including the operating system, its version, all
7 corresponding software and versions, and hardware information. Confirm you are
7 running within an environment that is a supported configuration, as many
7 problems can be explained by discovering software levels that are not meant to run
7 together, or have not been fully tested together.

7 **When does the problem happen?:**

7 Developing a detailed time line of events leading up to a failure is another
7 necessary step in problem analysis, especially for those cases that are one-time
7 occurrences. You can most easily do this by working backwards --start at the time
7 an error was reported (as exact as possible, even down to milliseconds), and work
7 backwards through available logs and information. Usually you only have to look
7 as far as the first suspicious event that you find in any diagnostic log, however this
7 is not always easy to do and will only come with practice. Knowing when to stop
7 is especially difficult when there are multiple layers of technology each with its
7 own diagnostic information.

- 7 • Does the problem only happen at a certain time of day or night?
- 7 • How often does this happen?
- 7 • What sequence of events lead up to the time the problem is reported?
- 7 • Does the problem happen after an environment change such as upgrading
7 existing or installing new software or hardware?

7 Responding to questions like this will help you create a detailed time line of
7 events, and will provide you with a frame of reference in which to investigate.

7 **Under which conditions does the problem happen?:**

7 Knowing what else is running at the time of a problem is important for any
7 complete problem description. If a problem occurs in a certain environment or
7 under certain conditions, that can be a key indicator of the problem cause.

- 7 • Does the problem always occur when performing the same task?
- 7 • Does a certain sequence of events need to occur for the problem to surface?
- 7 • Do other applications fail at the same time?

7 Answering these types of questions will help you explain the environment in
7 which the problem occurs, and correlate any dependencies. Remember that just
7 because multiple problems may have occurred around the same time, it does not
7 necessarily mean that they are always related.

7 **Is the problem reproducible?:**

7 From a problem description and investigation standpoint, the "ideal" problem is
7 one that is reproducible. With reproducible problems you almost always have a
7 larger set of tools or procedures available to use to help your investigation.
7 Consequently reproducible problems are usually easier to debug and solve.

7 However, reproducible problems can have a disadvantage: if the problem is of
7 significant business impact, you don't want it recurring. If possible, recreating the
7 problem in a test or development environment is often preferable in this case.

- 7 • Can the problem be recreated on a test machine?
- 7 • Are multiple users or applications encountering the same type of problem?
- 7 • Can the problem be recreated by running a single command, a set of commands,
7 or a particular application, or a standalone application?
- 7 • Can the problem be recreated by entering the equivalent command/query from
7 a DB2 command line?

7 Recreating a single incident problem in a test or development environment is often
7 preferable, as there is usually much more flexibility and control when
7 investigating.

7 Our example is reproducible.

7 **Conclusion:**

7 Describing a problem accurately and completely may be easy for some problems,
7 but very difficult for others. (The difficulty usually increases as the environmental
7 complexity increases). However, the questions that you need to ask are usually the
7 same: who, what, where, when, and how.

7 The db2support tool poses subsets of the questions described above when you
7 specify the "-q" option. This information is then used by DB2 Support to
7 understand the context in which you are encountering your problem. For example:
7 db2support . -q

7 Once have a good understanding of the problem situation, you can research
7 similar problems, and learn about workarounds, and fixes in the available DB2
7 troubleshooting resources.

7 **Related concepts:**

- 7 • on page 0

Identifying the version and service level of your product

The db2level command will help you determine the version and service level (build level and FixPak number) of your DB2 instance. To determine if your DB2 instance is at the latest service level, compare your db2level output to the build level and FixPak information on the FixPak download pages at the DB2 UDB support web site.

A typical result of running the db2level command on a Windows system would be:
C:\>db2level

```
DB21085I Instance "DB2" uses "32" bits and DB2 code release "SQL08020"
with level identifier "03010106".
Informational tokens are "DB2 v8.1.7.664", "s040914", "WR21342", and FixPak "7".
Product is installed at "D:\IBM\SQLLIB".
```

The combination of the four informational tokens uniquely identify the precise service level of your DB2 instance. This information is essential when contacting

IBM support for assistance. In particular, the second informational token shows the date that the product was built. In the above example, "s040914" indicates a build date of September 14th, 2004.

For JDBC or SQLJ application, if you are using the Universal Driver for SQLJ and JDBC, you can determine the level of the driver by running the db2jcc utility:

```
db2jcc -version
```

IBM DB2 JDBC Universal Driver Architecture 2.3.63

On Windows, this command is in the sqllib\bin directory.

On UNIX/Linux, this command is in the sqllib/bin directory.

DB2 server process model

Knowledge of the DB2® process model can help you determine the nature of a problem because it helps you to understand how the database manager and its associated components interact.

The process model used by all DB2 servers facilitates the communication that occurs between database servers and clients and local applications. It also ensures that database applications are isolated from resources such as database control blocks and critical database files.

UNIX-based environments use an architecture based on **system processes**. For example, the DB2 communications listeners are created as system processes. Intel operating systems such as Windows® use an architecture based on **threads** to maximize performance. Unless explicitly noted, this discussion uses the term "process" to refer to both processes and threads. You can find details of the differences between the use of Windows threads and UNIX® processes later in this topic.

For each database being accessed, various processes are started to deal with the various database tasks such as prefetching, communication, and logging.

Each process of a client application has a single **coordinator agent** that operates on a database. A coordinator agent works on behalf of an application, and communicates to other agents, using interprocess communication (IPC) or remote communication protocols.

DB2 architecture provides a **firewall** so that applications run in a different address space from DB2. The firewall protects the database and the database manager from applications, stored procedures, and user-defined functions (UDFs). A firewall maintains the integrity of the data in the databases, because it disables application programming errors from overwriting internal buffers or files of the database manager. The firewall also improves reliability, because application errors cannot crash the database manager.

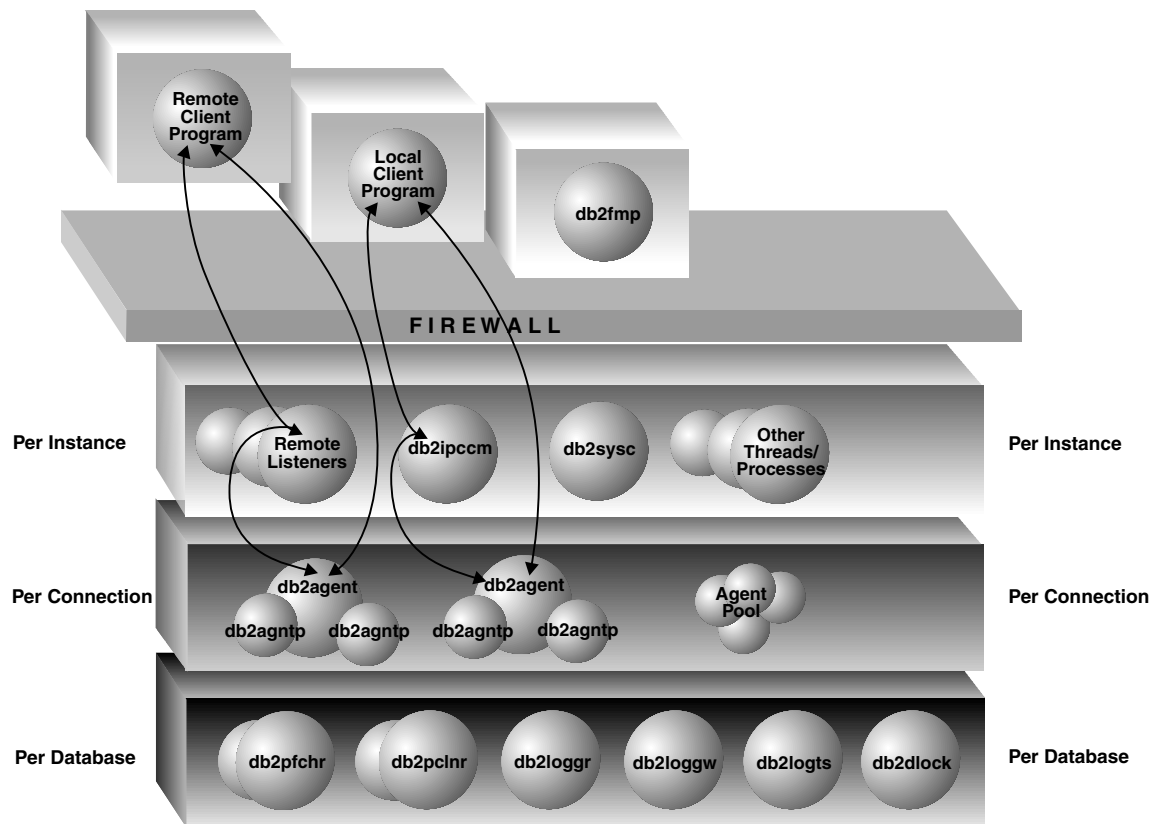


Figure 1. Process Model for DB2 Systems

The following list provides additional details on the processes shown in the figure:

Client Programs:

Client programs run remotely or on the same machine as the database server. They make their first contact with the database through a listener. A coordinator agent (db2agent) is then assigned to them.

Listeners:

Client programs make initial contact with communication listeners, which are started when DB2 is started. There is a listener for each configured communication protocol, and an interprocess communications (IPC) listener (db2ipccm) for local client programs. Listeners include:

- **db2ipccm**, for local client connections
- **db2tccm**, for TCP/IP connections
- **db2snacm**, for APPC connections
- **db2tcpdm**, for TCP/IP discovery tool requests

Agents:

All connection requests from client applications, whether they are local or remote, are allocated a corresponding coordinator agent (**db2agent**). When the coordinator agent is created, it performs all database requests on behalf of the application.

In some environments, in which the *intra_parallel* database manager configuration parameter is enabled, the coordinator agent distributes the database requests to subagents (**db2agntp**). These agents perform the requests for the application. Once the coordinator agent is created, it handles all database requests on behalf of its application by coordinating subagents (**db2agntp**) that perform requests on the database.

A coordinator agent may be:

- Connected to the database with an alias. For example, "db2agent (DATA1)" is connected to the database alias "DATA1".
- Attached to an instance. For example, "db2agent (user1)" is attached to the instance "user1".

An additional type of agent, **db2agnsc** is created by DB2 to perform certain operations in parallel. In particular, they are used in database recovery actions.

Idle agents reside in an agent pool. These agents are available for requests from coordinator agents operating on behalf of client programs, or from subagents operating on behalf of existing coordinator agents. The number of available agents is dependent on the database manager configuration parameters *maxagents* and *num_poolagents*.

db2fmp:

The fenced mode process. It is responsible for executing fenced stored procedures and user-defined functions outside the firewall. db2fmp is always a separate process but may be multithreaded depending on the types of routines it executes.

Database Threads/Processes:

The following list includes some of the important threads/processes used by each database:

- **db2pfchr**, for buffer pool prefetchers
- **db2pclnr**, for buffer pool page cleaners
- **db2loggr**, for manipulating log files to handle transaction processing and recovery
- **db2loggw**, for writing log records to the log files.
- **db2logts**, for collecting historical information about which logs are active when a tablespace is modified. This information is ultimately recorded in the DB2TSG.HIS file in the database directory. It is used to speed up tablespace rollforward recovery.
- **db2dlock**, for deadlock detection. In a multi-partitioned database environment, an additional process called **db2glock** is used to coordinate the information gathered from the db2dlock process on each partition. db2glock runs only on the catalog partition.

Database Server Threads and Processes:

The system controller (**db2sysc**) must exist in order for the database server to function. Also, the following threads and processes may be started to carry out various tasks:

- **db2resyn**, the resync agent that scans the global resync list

- **db2gds**, the global daemon spawner on UNIX-based systems that starts new processes
- **db2wdog**, the watchdog on UNIX-based systems that handles abnormal terminations
- **db2fcmdm**, the fast communications manager daemon for handling inter-partition communication (used only in multi-partitioned databases)
- **db2pdbc**, the parallel system controller, which handles parallel requests from remote nodes (used only in a partitioned database environment).
- **db2cart**, for archiving log files when accessing a database configured with USEREXIT enabled
- **db2fmtlg**, for formatting log files, when accessing a database configured with LOGRETAIN enabled, but with USEREXIT disabled
- **db2panic**, the panic agent, which handles urgent requests after agent limits have been reached at a particular node (used only in a partitioned database environment)
- **dlasync**, a monitor for the Data Links servers, if you have configured DB2 for datalinks

Differences between Windows and UNIX:

DB2 for Windows differs from UNIX-based environments in that the database engine is multi-threaded, not multi-processed. In Windows systems, each of the dispatchable units on the agent side of the firewall is a thread under the process db2sysc. This allows the database engine to let the operating system perform task-switching at the thread level instead of the process level. For each database being accessed, threads are started to deal with database tasks (for example, prefetching).

Another difference is in the handling of abnormal terminations. There is no need for a "watchdog" process in Windows systems, because these systems ensure that the allocated resources are cleaned up after an abnormal termination. Thus, there is no equivalent of the db2wdog process on the Windows systems. In addition, the db2gds process or thread is not needed on the Windows systems, which have their own mechanisms for starting threads.

Related concepts:

- on page 0
- "Determining active process status" on page 41

Proactive monitoring tools

Database monitoring is an important part of tuning a database server. You should develop a database monitoring plan. This not only allows for easier detection of a problem, but also enables you to refer back to historical data to isolate a change, extrapolate the effects of a change, better understand user and application activity and behaviors, and, of course, improve database performance!

Monitoring does involve overhead and this should be taken into account when developing the monitoring plan. The frequency and volume of monitoring needs to be controlled.

Techniques for monitoring the database:

DB2 has two main tools that allow you to access system monitor information, each serving a particular purpose:

- Snapshot monitoring - captures a picture of the database status at a particular point in time (the moment the snapshot is taken). Snapshot monitoring is useful in determining the current state of the database and its applications.
- Event monitoring - provides information about specific database events which occur over a period of time. Event monitoring is useful in identifying database or application state changes. It is used to gather details about changes regarding different events such as connections, tables, statements, etc.

In order to allow for effective memory usage, it is possible to configure how much memory is used for database monitoring. The database system monitor heap size configuration parameter `MON_HEAP_SZ`, which is a database manager configuration parameter, specifies the amount of memory allocated on a DB2START and is used when monitoring takes place. Setting this parameter to 0 (zero) will not allow any monitoring to take place.

Note: You must have one of SYSADM, SYSCTRL, SYSMAINT, or SYSMON authority to perform much of the set-up related to monitoring.

Levels of monitoring:

The monitoring utilities collect information at different levels within the system:

- *Database Manager* - the monitor gathers details from the time the instance is started till the time it is stopped.
- *Database* - this level of monitoring begins on the first connection or activation, and will continue until deactivation or the last connection is terminated.
- *Application* - this level of monitoring begins when an application connects to the database and continues until it disconnects from the database.

You can control the type of data to collect. The type of information is classified as follows:

BUFFERPOOL ACTIVITY
LOCKS
SORTS
SQL STATEMENTS
TABLE ACTIVITY
TIMESTAMPS
UNIT OF WORK

So, for example, if you notice long lock waits, it would be logical to monitor the LOCK group in an attempt to isolate the problem.

Snapshot and event monitors

For both snapshot and event monitors you have the option of storing monitor information in files or SQL tables, viewing it on screen (directing it to standard-out), or processing it with a client application. What follows are brief overviews to get you started with snapshot monitors and event monitors. For more information, refer to the System Monitor Guide and Reference, or the "Monitoring" section of the Information Center.

Setting up snapshot monitoring

The DB2® monitor maintains a running tally of valuable system information. You can get a summary of system status at any time by issuing the GET SNAPSHOT command. The parameters of the GET SNAPSHOT command determine the level of detail, and the type of monitoring information returned.

Some of the statistics returned by this command provide point-in-time information, others provide cumulative information since the last reset monitor command was performed. An example of a cumulative counter would be "deadlocks detected". This holds the number of deadlocks since the last reset monitor command. An example of an event-specific field would be "lock list memory in use". This field would contain the number of bytes currently used for the lock list and can change from one iteration of a snapshot to the next, as applications connect and disconnect and obtain and release locks respectively.

As an example, try using the **sample** database to take a snapshot. From a CLP session, issue the following commands:

```
DB2 CONNECT TO sample
DB2 RESET MONITOR ALL
DB2 GET SNAPSHOT FOR DATABASE ON sample
DB2 "SELECT * FROM syscat.bufferpools"
DB2 GET SNAPSHOT FOR DATABASE ON sample
```

Creating an event monitor

The Event Monitor facility within DB2 also allows for a great deal of flexibility. The initial step is to first create an event monitor.

As an example, try creating an event monitor for the SAMPLE database:

```
DB2 CONNECT TO sample
DB2 CREATE EVENT MONITOR evmonex1 FOR STATEMENTS WRITE TO FILE
'c:\temp\testfile' MAXFILES 3 MAXFILESIZE 1024 NONBLOCKED APPEND
```

The above command will create an event monitor called evmonex1 to capture all statement events. A maximum of 3 files of 4MB each will be created. The file directory (in this example "c:\temp\testfile") does not have to exist at CREATE EVENT MONITOR time. However, if it does not occur by the time an event monitor is activated, an error will occur (i.e. SQL1614N An I/O error occurred when activating an event monitor. Reason code = "2". SQLSTATE=58030).

Activating an event monitor

To enable the newly created event monitor to collect information, you must activate it. For example:

```
DB2 SET EVENT MONITOR evmonex1 STATE 1
```

To see if an event monitor is active or inactive, issue the SQL function EVENT_MON_STATE in a query against the table,SYSCAT.EVENTMONITORS, for example:

```
DB2 SELECT evmonname, EVENT_MON_STATE(evmonname) FROM syscat.eventmonitors
```

A returned value of 0 indicates that the specified event monitor is inactive, and 1 indicates that it is active.

For more example commands, including how to disable and drop event monitors, see Collecting information about database system events.

Formatting an event monitor

The output from file or pipe event monitors (as opposed to table event monitors) are written out in an unreadable format. In order to make the output readable, you need to run a tool called `db2evmon`:

```
db2evmon -db <database alias> -evm <event monitor name>
```

As an exercise, try generating and then formatting the output of the `evmonex1` event monitor created earlier.

```
db2evmon -db sample -evm evmonex1
```

Try running that command again after generating a statement event, for example by issuing

```
'DB2 "SELECT * FROM EMPLOYEE"'
```

. There will now be much more information, since an event has been triggered.

Basic monitoring commands

The following commands track current monitor settings, enabling functional groups, and "clear out" existing monitor data. These commands can be executed from a DB2 CLP session:

To obtain the current state of the functional groups listed earlier, issue the following command:

```
DB2 GET MONITOR SWITCHES
```

If you see that a particular group's "switch" is "on", you know that this group will be monitored.

Note: Event monitors are only affected by the time and timestamp information switch. All other switch settings effect snapshot monitors only and have no effect on the data collected by event monitors.

To enable or disable the monitor switches, use the command:

```
DB2 UPDATE MONITOR SWITCHES USING <switch name> ON | OFF
```

For partitioned database systems, you can set monitor switches specifically for a certain partition, or globally for all partitions. To set a monitor switch for a specific partition, issue the following command:

```
db2 update monitor switches using <switch name> ON|OFF at dbpartitionnum <partition number>
```

To set a monitor switch for all partitions, issue the command:

```
db2 update monitor switches using <switch name> on global
```

To reset monitor elements to zero, thereby allowing you to start with fresh system monitor information, use the following command (which can also be qualified to occur only on particular partitions):

```
DB2 RESET MONITOR ALL | FOR DATABASE <database alias>
```

Related reference:

- Snapshot monitor
- Snapshot monitor CLP commands
- Event monitors
- Creating an event monitor